# A Scalable Massive MIMO Uplink Baseband Processing Generator

Yue Dai, Greg LaCaille, Harrison Liew, James Dunn, Borivoje Nikolić
*Department of EECS, University of California, Berkeley, CA, USA*
{yuedai96, greglac, harrisonliew, jhdunn, bora}@berkeley.edu

ICC 2021 - IEEE International Conference on Communications | 978-1-7281-7122-7/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICC42927.2021.9500566

***Abstract*—This paper describes a scalable, highly portable, and power-efficient generator for massive multiple-input multiple-output (MIMO) uplink baseband processing. This generator is written in Chisel, and produces hardware instances for the distributed processing in a scalable massive MIMO system. The generator is parameterized in both the MIMO system and hardware datapath elements. The performance of several generator instances with different parameter values are validated by emulation on a field-programmable gate array (FPGA), demonstrating both functionality and scalability, and operation up to 6.4Gb/s data throughput.**

***Index Terms*—massive MIMO, generator, beamforming, FPGA-emulation**

## I. Introduction

With the increasing number of mobile devices and the increase of consumption and production of media-rich content, the demand for higher data rate is growing dramatically. Innovations in the capacity-achieving codes and efficient modulation techniques have brought spectral efficiency of wireless point-to-point systems close to the theoretical Shannon limit [1]. MIMO wireless technology is widely considered as an energy-efficient, secure and robust approach to increasing the overall channel capacity and to reducing the effects of interference, [2], [3]. By increasing the spatial resolution with hundreds to thousands of antennas at the base station (BS), massive MIMO can support numerous users in the same time-frequency resource by providing each user with their interference-free, high-capacity link to the base station (BS) [4]. The integration of many antennas can be enabled by operating at millimeter-wave (mm-wave) frequencies due to the small antenna size. [5].

While the massive MIMO is attractive, implementing the system in a cost-efficient and energy-efficient way is challenging. Some designed testbeds, for example, Argos [6], use the centralized processing, where all data are collected from antennas and processed in the centralized processor. The LuMaMi is another massive MIMO testbed which implements the centralized processing on co-processors [7]. Those architectures highly depend on interconnection bandwidth and require complex, high-throughput, router designs. To solve the interconnection bottleneck and improve design scalability, the massive MIMO system has to be distributed. An implementation of the decentralized baseband processing for a massive MIMO system in the graphic processing unit (GPU) has been proposed by Li [8], which partitions antennas into clusters and detects data locally to reduce inter-cluster communication bandwidth. Although the GPU and the software-defined radio (SDR) enable flexibility for experimentation with algorithms, they are not a practical power-efficient solution.

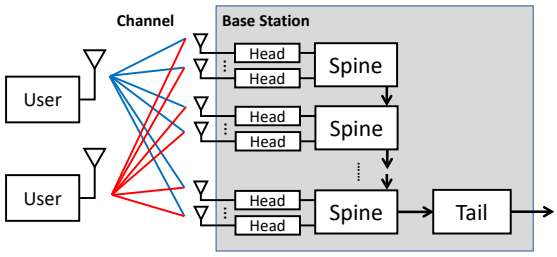



Fig. 1. The scalable massive MIMO BS architecture for uplink.

Energy efficiency can be achieved through hardware specialization; On the other hand, designing the specialized hardware is costly. The approach of designing hardware generators, as opposed to instances, enables reuse and thus reduces design cost [9], [10]. For many signal processing tasks, reusing the algorithm implementation enables faster development of application-optimized hardware [9], [11].

The paper is organized as follows. Section II gives the overview of the scalable massive MIMO uplink baseband processing system. Section III discusses the design of the generator for the distributed processing part in the system. Section IV details the "golden" model simulator and FPGA emulation setup for the generator evaluation. The functionality and performance are demonstrated and analyzed in Section V.

## II. System Overview

Fig. 1 shows the uplink BS architecture. The BS consists of three parts: Heads, Spines, and the Tail. The Head is the mm-wave (or radio frequency) front-end. The Spine contains a maximum-ratio combining (MRC) beamformer for a sub-array. All Spines are chained up through the daisy chain architecture. The Tail receives the MRC beam-formed signal, and performs the frequency-selective decorrelation to remove inter-user interference.

### A. Two-stage beamforming

Linear beamforming for wideband channels needs to consider the frequency-selective propagation effect. To improve

978-1-7281-7122-7/21/$31.00 ©2021 IEEE
Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:29 UTC from IEEE Xplore. Restrictions apply.

energy-efficiency of computation, a two-stage beamforming algorithm has been proposed [12].

*1) MRC beamformer:* Theoretically, in flat-fading channels, MRC can maximize the output SNR [13], which is the purpose of the first stage beamformer. Assume that a multi-user MIMO (MU-MIMO) system containing $K$ users and $M$ antennas at the BS; let $\mathbf{H} \in \mathbb{C}^{M \times K}$ be the channel matrix, $\mathbf{s} \in \mathbb{C}^{K}$ be the transmitted symbols from users and $\mathbf{y} \in \mathbb{C}^{M}$ be the signal received at the BS. Then

$$\mathbf{y} = \mathbf{H}\mathbf{s} + \mathbf{n}, \tag{1}$$

where $\mathbf{n} \sim \mathcal{CN}(0, \sigma^2 \mathbf{I}_M)$ is independently and identically distributed (i.i.d) zero mean complex Gaussian noise. Let $\tilde{\mathbf{y}}_{MRC} \in \mathbb{C}^{K}$ be the MRC beamformed signal at BS. Then the MRC beamformer gives

$$\tilde{\mathbf{y}}_{MRC} = \mathbf{H}^H \mathbf{y} = \mathbf{H}^H \mathbf{H}\mathbf{s} + \mathbf{H}^H \mathbf{n}. \tag{2}$$

The MRC beamformer can be split into multiple subarrays and then forms a distributed system by computing each sub-block locally, followed by summing the results through the daisy chain. Let $N$ equal to the number of sub-arrays, $\mathbf{B}_i \in \mathbb{C}^{M/N \times K}$ where $0 < i < N$, $\mathbf{H} = [\mathbf{B}_1^H \ \mathbf{B}_2^H \ \ldots \ \mathbf{B}_N^H]^H$, and $\mathbf{y}_i \in \mathbb{C}^{M/N}$ is the received signal at $i$-th sub-array. The previous equation can be expressed as

$$\tilde{\mathbf{y}}_{MRC} = \sum_{i=1}^{N} \tilde{\mathbf{y}}_{i,MRC} = \sum_{i=1}^{N} \mathbf{B}_i^H \mathbf{y}_i. \tag{3}$$

*2) Frequency-selective decorrelation:* For a MU system, the inter-user and inter-symbol interference are still present after the MRC beamforming. At this stage, a wideband channel is translated into multiple narrow subcarriers, and each narrow subcarrier performs the zero-forcing algorithm. Let $N_s$ be the number of samples of the OFDM pilot, $N_g$ be the number of subcarriers. Then each subcarrier contains $N_s/N_g$ samples. Let $\tilde{\mathbf{Y}}_{i,MRC} = FFT(\tilde{\mathbf{y}}_{MRC})[i]$ be the $i$-th sample of the MRC beam-formed signal in the frequency domain, and $\mathbf{H}_i$ be the channel matrix for the $i$-th sample. The zero-forcing estimator gives

$$\hat{\mathbf{x}}_i = IFFT((\mathbf{H}_i^H \mathbf{H}_i)^{-1} \tilde{\mathbf{Y}}_{i,MRC}). \tag{4}$$

The estimation of $\mathbf{H}_i$ is the average of all estimations of $\mathbf{H}_j$ within the same subcarrier.

After the two-stage beamforming, the inter-user interference is removed and signal-to-interference-plus-noise ratio (SINR) increases.

### B. Signal packet format

Fig. 2 shows the individual user's signal packet and the signal packet received by each BS channel, respectively. Users send time-interleaved Golay pilots first, followed by time-interleaved OFDM pilots, to help perform the channel estimation in the two-stage beamformer. Golay pilots are used for the estimation of a frequency-flat channel matrix in the MRC beamformer, which consists of Golay complementary pairs. The BPSK-modulated OFDM pilots are used for the frequency-selective

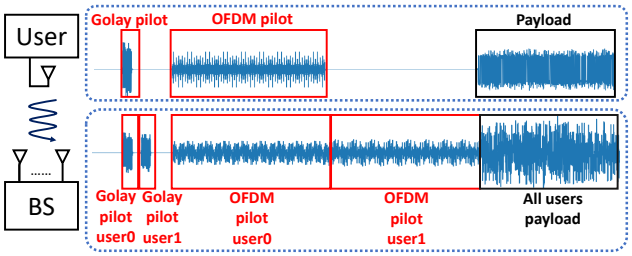



Fig. 2. The signal packet format. The upper inset is the user TX signal packet. The lower inset is each BS channel received signal packet.

channel estimation in the decorrelation stage. Both Golay pilots and OFDM pilots include guard intervals. After all users send both pilots, they send quadrature amplitude modulated (QAM) payloads simultaneously. To synchronize the BS and users, a beacon signal is propagated to all users from the BS at the beginning of each packet to work as a time reference.

## III. The Spine Generator

Generators are modular, highly parameterizable register-transfer level (RTL) hardware designs implemented in the higher-level programming language to enable parametrization and design reuse via extension. The Spine generator is written in Chisel [14], which is a hardware construction language that facilitates parameterized circuit generation for both ASIC and FPGA digital logic designs. It generates the MRC beamformer stage in this scalable massive MIMO BS architecture. The Spine generator is parameterized in the number of channels per Spine, the number of users in the system, the oversampling rate, the number of root-raised cosine (RRC) filter taps, the Golay pilot design, the Lagrange polynomial order, the datapath bitwidth, and the datapath parallelization. It is integrated with multiple generators, which are designed for the general digital signal processing (DSP) by using the reusable hardware library described in [15]. The Spine datapath is shown in Figure 3. The inputs are $M$ channels of in-phase and quadrature (IQ) signals which are parallelized at the configuration time. The outputs are $K$ users MRC beam-formed signal summed with the signal coming from the upper neighboring Spine in the daisy chain. The outputs are sent to the lower neighboring Spine. The parameters of each module generator are shown in the yellow boxes.

### A. Signal Correction

The signal correction group includes a 65-tap RRC filter, an IQ signal synchronizer, and a direct current (DC) offset cancellation for each channel. The RRC filter is generated by the tree-reduced strength-reduced FIR filter generator. The number of taps is parameterizable, and the coefficient array is configurable. To balance the front-end impairments, the IQ synchronizer and the DC offset cancellation are included in the datapath set with manually calibrated IQ impairment and DC.

### B. Frequency-flat channel estimation

The MRC beamforming stage and the frequency-selective decorrelation stage perform the channel estimation separately.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:29 UTC from IEEE Xplore. Restrictions apply.

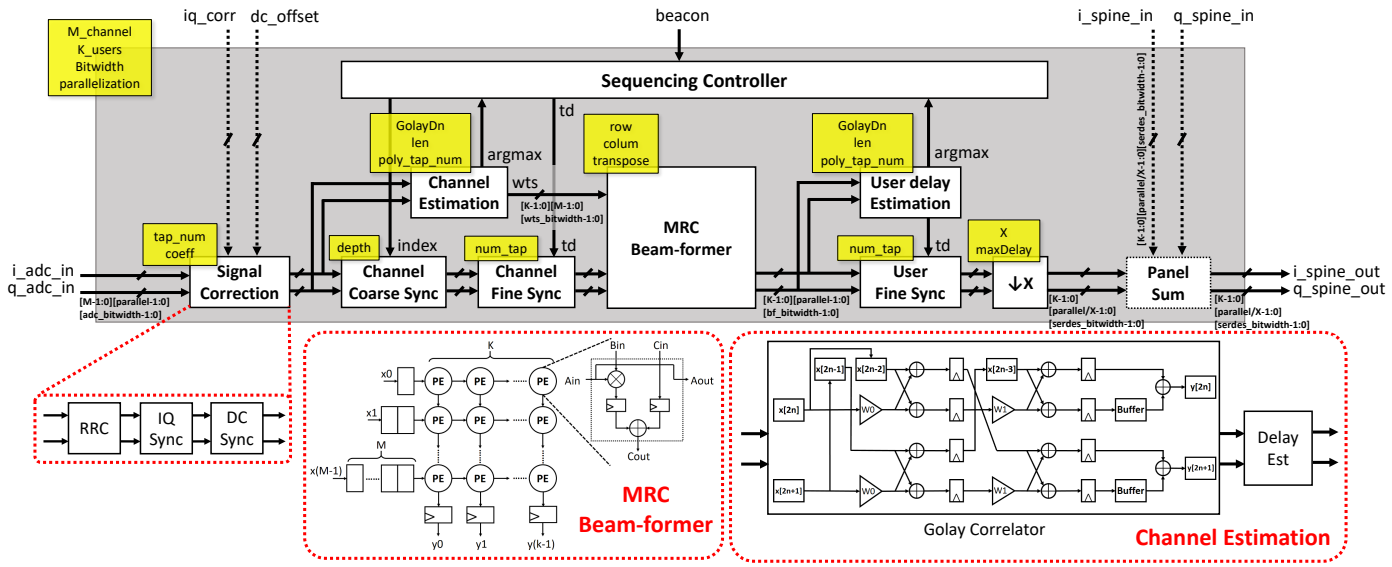



Fig. 3. The Spine generator datapath.

In the MRC beamformer stage, the Golay pilot is used for the frequency-flat channel estimation; while in the frequency-selective decorrelation stage, the OFDM pilots aid the channel estimation. The detailed design of the frequency-selective channel estimation can be found in [12].

The Golay pilot contains two complementary sequences with the same length $L$, where $L$ is a power of 2. The sum of the aperiodic autocorrelation functions of the sequence pair is a Kronecker delta [16]. Let $h_{m,k}$ represent the $m$th-row $k$th-column element in the channel matrix $\mathbf{H}$, and $\mathbf{g}_k$ be the Golay pilot of user k. Because the Golay pilot is time-interleaved, the pilot received at the channel $m$ is given by $\tilde{\mathbf{g}}_{k,m} = h_{m,k}\mathbf{g}_k + \mathbf{n}$, where $\mathbf{n}$ is the additive white Gaussian noise. Let the autocorrelation of $\tilde{\mathbf{g}}_{k,m}$ be $R_{k,m}$. The estimation of $h_{m,k}$ is

$$\hat{h}_{m,k} = \frac{1}{2L} R_{k,m}. \tag{5}$$

According to [16], the Golay sequence with length $2L$ can be generated by a sequence of delayed values $\mathbf{D}$ and a sequence of seeds $\mathbf{W}$ with length $log_2(L)$. In this design, $\mathbf{D}$ is any permutation of the set $\{2^0, 2^1, ....., 2^{log_2(L)-1}\}$, and the seed $w \in \{-1, 1\}$. We propose the pipelined parallelizable Golay correlator generator based on the algorithm in [16]. The proposed design doesn't contain multipliers, which makes it energy-efficient. The datapath is pipelined by $log_2(L)$ stages. The amount of parallelism changes the delay register connection and the input of the adder. The adder input can be either from the delay registers or the pipeline registers. The architecture is hardware parameterized for $\mathbf{D}$, but can be configured for $\mathbf{W}$ on the fly. The sequencing controller controls the input of $W$ at each user's Golay pilot time interval. The channel estimation inset in Fig. 3 shows the generated architecture of an oversampling-2, length-4 Golay correlator.

The channel delay estimation contains two parts: coarse delay estimation and fine delay estimation. The coarse delay estimation is based on the correlation norm peak of Golay pilots in Eq. 5. However, due to the ADC skew and the sampling phase mismatch, the signal may not be sampled at the maximum power, which does not maximize the correlation norm peak. To resample the signals with maximum power, Lagrange polynomial $\mathcal{L}(x)$ is used to interpolate the correlation norm based on $2l+1$ correlation norm samples $c(i)$ around the peak, where $i \in \{-l, ..., 0, ...l\}$. The fine delay estimation $\hat{s}_0$ is

$$\hat{s}_0 = \arg\max_x \mathcal{L}(x) = \arg\max_x \sum_{i=-l}^{l} \left[ c(i) \prod_{-l, j\neq i}^{l} \frac{x-j}{i-j} \right]. \tag{6}$$

Let $y(t)$ be the skewed signal and $\hat{y}(t)$ be the deskewed signal, which is given by

$$\hat{y}(t) = \sum_{i=-l}^{l} \left[ y(t+i) \prod_{-l, j\neq i}^{l} \frac{\hat{s}_0 - j}{i-j} \right]. \tag{7}$$

### C. MRC beamformer

The MRC beamformer is generated by the pipelined systolic-array-based matrix multiplier generator, with parameterized matrix dimensions and datapath parallelization. The matrix multiplier uses the weight-stationary architecture. Each processing element (PE) consists of a width-parameterized complex, fixed-point multiplier and an adder. For a massive MIMO system with $K$ users and $M$ channels per Spine, the matrix multiplier dimension is $M \times K$ for a single path. In order to meet timing requirement for different platforms or technology, the number of pipeline stages along $K$ dimension is also parameterized.

The lower middle inset in Fig. 3 shows the beamformer architecture. The weight inputs are the estimated frequency-flat channel matrix, and the input to each row is the channel's fine synchronized signal.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:29 UTC from IEEE Xplore. Restrictions apply.

### D. Sequencing Controller

The sequencing controller is designed for: 1) providing the time reference for each processing module; 2) channel synchronization.

Due to different user propagation delays and the front-end mismatches, the signal arrival times are mismatched, even for the same user. Let the time delay created by the user $i$ be $T_{u,i}$ and the time delay created by the channel $j$ be $T_{c,j}$ where $i \in [1, K]$ and $j \in [1, M/N]$. We can assume $T_u \sim \mathcal{N}(\mu_u, \sigma_u^2)$. Then the time delay between user $i$ and channel $j$ in the same Spine module is $T_{u,i} + T_{c,j}$. To synchronize the channel before the MRC beamformer, a channel time delay estimator is needed. This can be achieved by averaging the time delay among all users signal received by the same channel.

$$\mathbb{E}\Big[\frac{1}{K}\sum_{i=0}^{K}(T_{cj} + T_{ui})\Big] = T_{cj} + \mu_u. \tag{8}$$

The user time delay needs to be estimated after the MRC beamforming. The user time delay estimation uses the same algorithm as the channel delay estimation. The downsampling phase selection is determined by the user coarse time delay estimation.

## IV. Evaluation

### A. Simulator

A Python-based system simulator is built to simulate an end-to-end massive MIMO system, including the transmission packet generation, various channel models, front-end impairments, and modular baseband signal processing [12]. We treat this simulator as a "golden" model and evaluate the performance of this generator against it.

The transmission packet is parameterized in terms of the QAM modulation order, the pilot length, the guard interval length, and the number of sub-carriers for the OFDM pilot.

Since the Spine generator implements the frequency-flat MRC beamformer, the performance is evaluated on: 1) frequency-flat i.i.d channel; 2) Rician channel with $K = 10dB$.

The reference simulator includes the Spine and the Tail modules. The Spine algorithm shown in Fig. 3 is simulated in floating-point data format. The Tail is simulated with the frequency-selective decorrelation algorithm on the beamformed signal to eliminate inter-user interference and equalize inter-symbol interference [12].

### B. FPGA Emulation

The Spine generator is implemented on a Xilinx VCU118 FPGA. The FPGA emulation system design is shown in Fig. 4. The FPGA contains: 1) the Spine digital signal processing (DSP) core generated by the Spine generator but without the panel summation module; 2) the memory system; 3) and the clock generators for two clock domains. The server transfers the data to the FPGA, and simultaneously configures the Spine DSP core and DMA through JTAG. Because the panel summation module does not lose precision, the emulation performance should be the same as if the summation is implemented on the

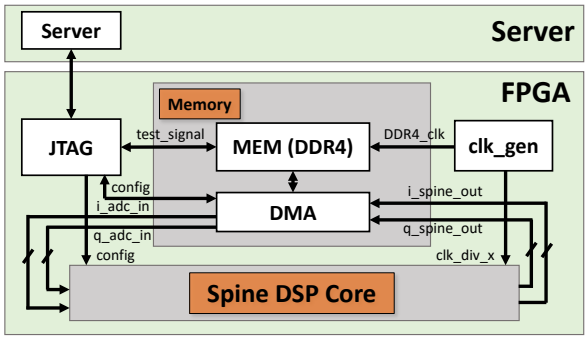



Fig. 4. The scalable massive MIMO uplink baseband processing system emulation architecture.

TABLE I
FPGA EMULATION SYSTEM PARAMETER

| | Parameter | Value |
|---|---|---|
| System parameter | MIMO system parameter | 32 channels, 2 users |
| | Signal bandwidth | 200MHz |
| | Oversampling rate | 2 |
| | Modulation scheme | QPSK, 16-QAM |
| FPGA parameter | FPGA Type | Xilinx VCU 118 |
| | Number of channels per Spine | 4 |
| | Datapath bitwidth | 8 |
| | Datapath parallelization | 8 |
| | Baseband clock freq | 50MHz |

FPGA. The emulation system parameters are shown in Table. I.

The emulation data flow is shown in Fig. 5. The Python simulator generates user packets and simulates the channel. The Tcl script generator converts data to the 8-bit fixed-point format, and generates the Tcl scripts for the data transmission control and the Spine DSP core configuration. After the FPGA finishes data processing, the data are loaded from the memory back to the server through JTAG. The emulation data are then reformatted to the floating-point format and simulated by the Tail simulator.

## V. Results

### A. Performance

The Spine generator demodulation bit-error rates (BER) are evaluated under different SNR scenarios. The channel models are the flat i.i.d channel and the Rician channel with $K = 10dB$. The results of BER vs. SNR for the FPGA emulation system is shown in Fig. 6. The figure shows the BER ranging from $10^{-6}$ to $10^{-3}$. The inset a) shows the BER of the quadrature phase-shift keying (QPSK) modulation, and the inset b) shows that of the 16-QAM. The result shows that at SNR 11.9dB for the QPSK modulation and 19dB for the 16-QAM modulation, the BER reaches $10^{-3}$ in the flat i.i.d channel. For both QPSK and 16-QAM modulation scheme, the emulation BER is very close to the simulation BER produced by the floating-point golden model, validating the functionality of the Spine generator. The small difference between the simulation

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:29 UTC from IEEE Xplore. Restrictions apply.

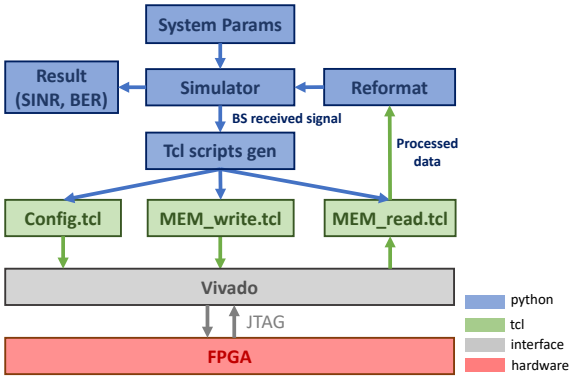



Fig. 5. Emulation system control flow chart.

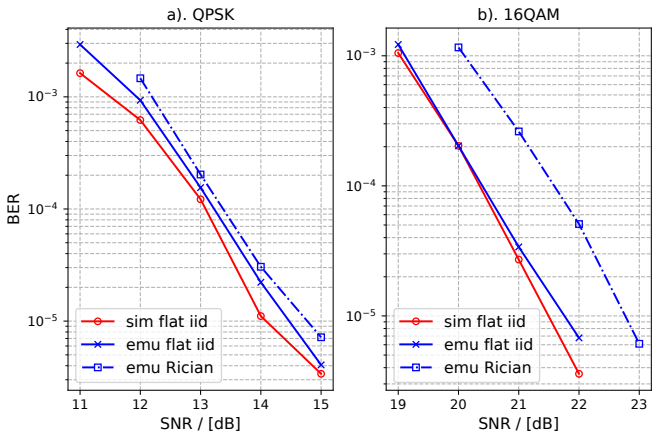



Fig. 6. BER vs SNR for 32 antenna 2 users simulation and emulation results for a). QPSK modulation scheme and b). 16-QAM scheme.

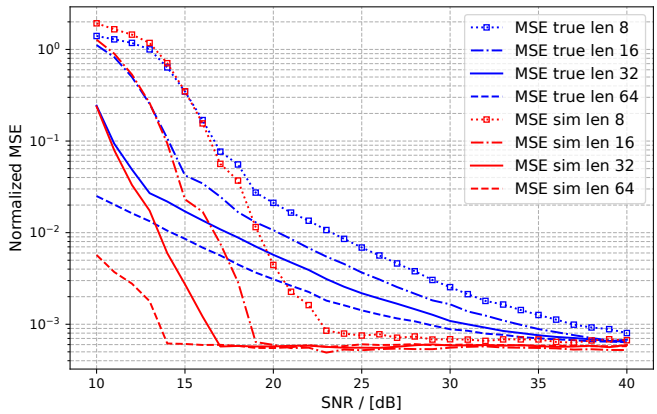



Fig. 7. Frequency flat normalized channel estimation MSE vs SNR with different Golay pilot length. Blue lines show the normalized MSE of the true channel matrix and the channel estimation in the FPGA; red lines show the normalized channel estimation MSE of the simulator and FPGA.

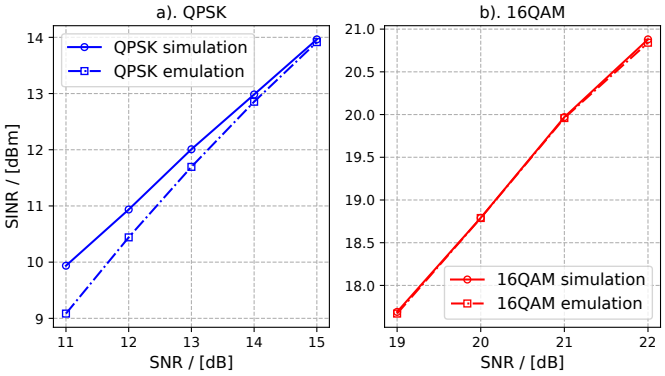



Fig. 8. SINR versus SNR for 32 antenna 2 users system simulation and emulation results for a). QPSK modulation scheme and b). 16-QAM scheme.

and emulation performance comes from the residual channel estimation and the datapath quantization errors.

The blue lines in Fig. 7 show the normalized mean-square error (MSE) of the true channel matrix and the frequency-flat channel estimation in the FPGA versus SNR for different Golay pilot lengths in the flat i.i.d channel. The normalized MSE of the channel estimations decreases as the Golay pilot length increases. From the figure, one can also notice that when the SNR is lower than 17 dB, 15 dB, and 13dB for the lengths of 8, 16, and 32, respectively, the MSE increases faster than higher SNR in log scale. The change of the slope is due to the increase of pilot synchronization failure. However, by increasing the Golay pilot length to 64, the channel estimation MSE reduces to lower than $3 \times 10^{-2}$ for the SNR even lower than 15dB, and the possibility of pilot synchronization failure becomes much lower.

The red lines in Fig. 7 show that when the SNR is lower than 23 dB, 19 dB, 17 dB, and 13dB for the pilot length of 8, 16, 32, 64, respectively, the normalized channel estimation MSE of the simulator and the FPGA begins to increase. The normalized MSE converges to $5 \times 10^{-4}$. To fit into the 8-bit datapath, a 0.2 gain is applied at the emulation input. Then the normalized MSE floor corresponds to about 0.5 quantization bit, which is the quantization error. Emulated design adopts the Golay pilot length of 64 because of its adequately low MSE. In Fig. 8, the SINR difference for the QPSK slightly increases as the SNR decreases, which agrees with the MSE measurements from the channel estimation. For 16-QAM, the SINR for the simulation and emulation is nearly the same.

Fig. 6 also shows the BER versus SNR for the QPSK and 16-QAM in a low-multipath Rician channel. For the QPSK modulation, the performance is similar to the flat i.i.d channel. For the 16-QAM modulation, the performance degrades by about 1 dB. Compared with the QPSK modulation, the 16-QAM modulation is more sensitive to the imperfect channel estimation in the Rician channel model, which leads to the larger performance degradation than that of the QPSK modulation.

Emulation time for all presented BER measurements is 64 hours.

### *B. Throughput*

The throughput of the Spine DSP core depends on the number of users in the system $K$, baseband clock frequency $f$, oversampling rate $x$, datapath bitwidth $b$, and datapath parallelization $p$. Because both I and Q have the same bitwidth, the throughput of a single Spine is $2fbpK/x$ bits per second. By

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:29 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON

| | This work | LuMaMi [7] | RIVF'19 [17] | JETCAS'17 [8] |
|---|---|---|---|---|
| Platform | FPGA/ASIC | FPGA | FPGA | GPU |
| Portable | Yes | No | No | No |
| Modulation | QAM | OFDM | QAM | OFDM |
| Distributed | Yes | No | No | Yes |
| Bandwidth | 200MHz[1] | 20MHz | 935MHz[2] | 40MHz |
| Method | Two-stage BF | MRC/ZF/RZF | SOR | ADMM |
| CHEST[3] | Yes | Yes | No | Yes |
| Power | 5.4W[1] | - | - | 235W[4] |

[1] The signal bandwidth is the FPGA implementation bandwidth. The power estimation are based on the FPGA implementation of a single Spine with parameters shown in Table I.
[2] This is the maximum FPGA implementation clock frequency, not the signal bandwidth. The signal bandwidth isn't given.
[3] Short for channel estimation.
[4] This is the thermal design of the Tesla K40 GPU with fully utilization.

increasing datapath parallelism, which is a generator parameter, the throughput can scale up. For the specific implementation in this paper, the throughput is 6.4Gb/s.

Notice that due to the daisy chain connection of Spines, the final throughput of the MRC beamforming stage doesn't scale up with the number of channels in the system, and is the same as a single Spine. Compared with the LuMaMi whose interconnection bandwidth is proportional to the number of antennas at the front-end [7], our work sigifficantly relaxes the interconnect requirements.

### C. Power estimation

The estimated power for each single Spine FPGA implementation is about 5.4W. This design can be mapped onto an ASIC for further power savings.

### D. Summary

Table II summarizes several state-of-the-art massive MIMO BS implementations. Unlike [17] and [7] which include proprietary external FPGA modules, the design, the Spine generator is written in Chisel and is open-source [18]. This leads to its high portability to different hardware platforms, either ASIC or FPGA, by easily changing the system and hardware parameters. Compared to other narrowband OFDM designs in [7] and [8], our work supports much higher signal bandwidth, since the bandwidth of the carrier recovery algorithm in the Tail is not limited by the OFDM subcarrier bandwidth by using QAM payloads. The distributed processing and daisy chain architecture in our work relaxes the interconnection requirements and improves the system scalability. Compared with the GPU implementation in [8] which involves amount of memory access overhead, our work is specialized and much more power-efficient.

## VI. CONCLUSION

This paper presents the design and the emulation results of a scalable, highly portable, and power-efficient massive MIMO uplink baseband processing generator - the Spine generator. The Spine generator is parameterized across a range of MIMO system and the datapath hardware parameters, and specific instances were emulated on the FPGA. The BER and SINR under different channels with different modulation schemes are evaluated. The parallelization and pipelining of the configurable datapath and the distributed processing architecture enable the high throughput of the Spine without putting pressure on the clock frequency or scaling up with the number of channels in the system.

## ACKNOWLEDGMENT

This work was supported in part by tasks 2778.026 and 2778.007 of the ComSenTer Research Center, one of six centers in JUMP, a Semiconductor Research Corporation (SRC) program sponsored by DARPA, and in part by NSF EARS award 1642920. The authors also acknowledge the students, staff, faculty, and sponsors of the Berkeley Wireless Research Center.

## REFERENCES

[1] A.Goldsmith, "5g and beyond: What lies ahead for wireless system design," in *PIMRC*, 2014.
[2] E. G. Larsson *et al.*, "Massive mimo for next generation wireless systems," *IEEE Communications Magazine*, vol. 52, no. 2, pp. 186–195, 2014.
[3] F. Rusek *et al.*, "Scaling up mimo: Opportunities and challenges with very large arrays," *IEEE Signal Processing Magazine*, vol. 30, no. 1, pp. 40–60, 2013.
[4] T. L. Marzetta, "Noncooperative cellular wireless with unlimited numbers of base station antennas," *IEEE Transactions on Wireless Communications*, vol. 9, no. 11, pp. 3590–3600, 2010.
[5] S. A. Busari *et al.*, "Millimeter-wave massive mimo communication for future wireless systems: A survey," *IEEE Communications Surveys Tutorials*, vol. 20, no. 2, pp. 836–869, 2018.
[6] C. Shepard *et al.*, "Argos: practical many-antenna base stations," *MobiCom*, 2012.
[7] S. Malkowsky *et al.*, "The world's first real-time testbed for massive mimo: Design, implementation, and validation," *IEEE Access*, vol. 5, pp. 9073–9088, 2017.
[8] K. Li *et al.*, "Decentralized baseband processing for massive mu-mimo systems," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 7, no. 4, pp. 491–507, 2017.
[9] B. Nikolic *et al.*, "Generating the next wave of custom silicon," in *IEEE 44th European Solid-State Circuits Conference*, 2018, pp. 6–11.
[10] B. Nikolić, "Simpler, more efficient design," in *IEEE 41st European Solid-State Circuits Conference*, 2015, pp. 20–25.
[11] O. Shacham *et al.*, "Rethinking digital design: Why design must change," *IEEE Micro*, vol. 30, no. 6, pp. 9–24, 2010.
[12] G. LaCaille *et al.*, "Design and demonstration of a scalable massive mimo uplink at e-band," in *2020 IEEE International Conference on Communications Workshops*, 2020, pp. 1–6.
[13] K. S. Ahn *et al.*, "Performance analysis of maximum ratio combining with imperfect channel estimation in the presence of cochannel interferences," *IEEE Transactions on Wireless Communications*, vol. 8, no. 3, pp. 1080–1085, 2009.
[14] J. Bachrach *et al.*, "Chisel: Constructing hardware in a scala embedded language," in *Design Automation Conference*, 2012, pp. 1212–1221.
[15] A. Wang *et al.*, "Aced: A hardware library for generating dsp systems," in *Proceedings of the 55th Annual Design Automation Conference*, ser. DAC '18, 2018.
[16] E. García *et al.*, "Efficient filter for the generation/correlation of golay binary sequence pairs," *International Journal of Circuit Theory and Applications*, vol. 42, no. 10, pp. 1006–1015, 2014.
[17] C. Nhat Cuong *et al.*, "Hardware implementation of the efficient sor-based massive mimo detection for uplink," in *2019 IEEE-RIVF International Conference on Computing and Communication Technologies*, 2019, pp. 1–6.
[18] "Chisel3," https://github.com/chipsalliance/chisel3. Accessed on 2020-02-20.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:29 UTC from IEEE Xplore. Restrictions apply.